Healthcare Analytics Pipeline for MediCare Hospital

Submitted By:

Arpit Sharma — Roll No: G25AI1012

Ankita Dwivedi — Roll No: G25AI1011

Astosh Ranjan — Roll No: G25AI1013

Bharat Sharma — Roll No: G25AI1014

Damini Khatri — Roll No: G25AI1015

Course: Post Graduate Diploma in Data Engineering (PGDDE)

Institute: IIT Jodhpur

Plagiarism Declaration

We declare that this project report, implementation, configurations, scripts, workflows, dashboards, database designs, and deployment procedures are the original work of our team carried out during academic practice.

All necessary references, documentation sources, and publicly available resources used during the implementation have been properly acknowledged. Any form of plagiarism may lead to disciplinary action as per academic policies.

PART 1 — OBJECTIVE

The objective of this project is to:

• Build an end-to-end Healthcare Analytics Pipeline.

• Automate healthcare data ingestion, processing, and reporting.

• Store and manage healthcare data using cloud technologies.

• Perform large-scale healthcare analytics using PySpark.

• Design a healthcare data warehouse using Star Schema.

• Develop interactive dashboards for healthcare insights.

• Implement machine learning for predictive healthcare analytics.

This project demonstrates the practical implementation of Data Engineering, Cloud Computing, Big Data Processing, Data Warehousing, Analytics, and Visualization technologies in a healthcare environment.

PART 2 — PROBLEM STATEMENT

Healthcare organizations generate large volumes of patient, treatment, and operational data every day.

Traditional approaches:

• Require manual data processing and reporting.

• Are time-consuming and error-prone.

• Lack centralized data management.

• Struggle with large-scale data processing.

• Provide limited analytical capabilities.

This system addresses these challenges by:

• Automating data ingestion and processing.

• Centralizing healthcare data storage.

• Enabling scalable analytics using cloud infrastructure.

• Providing automated workflow execution.

• Delivering interactive dashboards and business insights.

PART 3 — DATASET DESCRIPTION

The project uses a healthcare dataset containing patient records and treatment information.

Dataset Characteristics:

• Total Records: 5000

• Data Format: CSV

• Domain: Healthcare Analytics

• Storage Platform: AWS S3

Dataset Attributes:

• Patient ID

• Patient Name

• Age

• Gender

• Disease

• City

• Treatment Cost

• Admission Status

• Admission Date

The dataset contains anonymized healthcare information and was used for ETL processing, analytics, data warehousing, dashboard development, and machine learning implementation.

PART 4 — TECHNOLOGY STACK

The following technologies were used during project implementation:

Component	**Technology Used**
Programming Language	Python
Data Processing	Pandas
Workflow Automation	Apache Airflow
Big Data Processing	PySpark, Databricks
Data Streaming	Apache Kafka
Cloud Storage	AWS S3
Database	PostgreSQL
Cloud Database	AWS RDS PostgreSQL
Data Warehouse	Star Schema
Dashboard	Streamlit
Machine Learning	Scikit-Learn, Random Forest
Version Control	Git & GitHub
CI/CD	GitHub Actions
Containerization	Docker

The selected technology stack provides scalability, automation, cloud integration, and analytical capabilities required for modern healthcare data engineering solutions.

PART 5 — IMPLEMENTATION

The Healthcare Analytics Pipeline was implemented using a modular architecture consisting of cloud storage, ETL processing, workflow automation, database integration, big data processing, analytics, and visualization components.

Step 1 — AWS S3 Data Storage

Healthcare records were stored in AWS S3 cloud storage before processing.

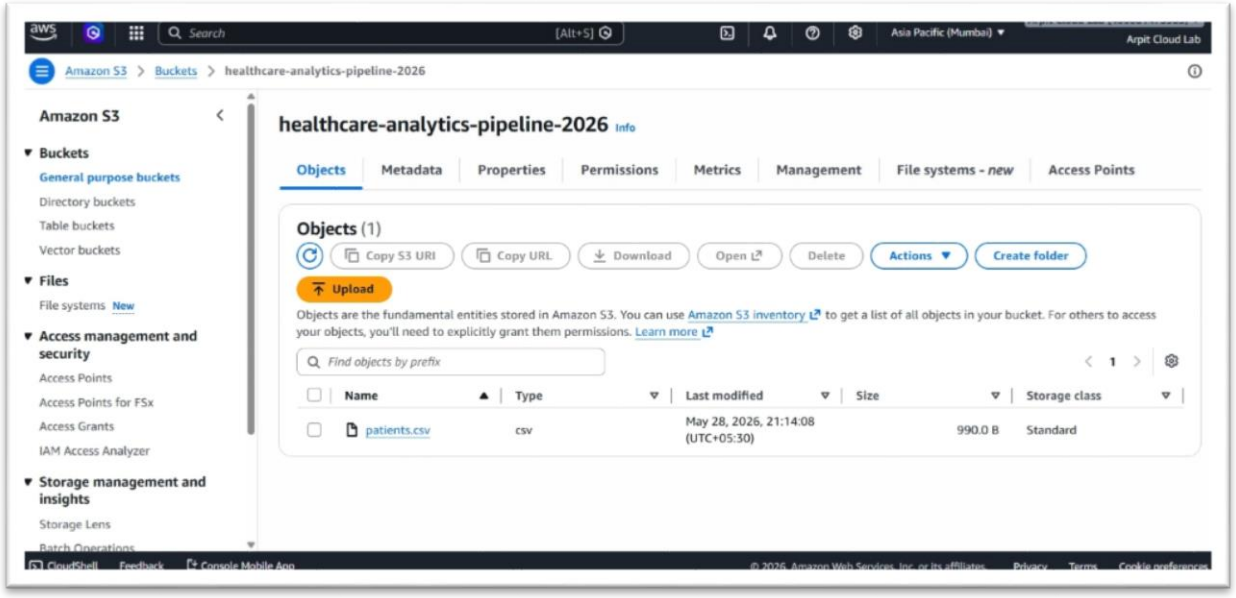
Benefits:

• Centralized data storage

• Cloud-based accessibility

• Scalability for future data growth

• Integration with ETL workflows



Step 2 — ETL Pipeline Implementation

An ETL (Extract, Transform, Load) pipeline was developed using Python and Pandas.

Activities Performed:

• Dataset extraction

• Data cleaning

• Missing value handling

• Data validation

• Data transformation

• Loading into PostgreSQL

The ETL pipeline automated healthcare data processing and reduced manual effort.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline> cd scripts
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline\scripts> python etl_pipeline.py
Healthcare Dataset Loaded Successfully
  patient_id          name  age  gender       disease admission_date discharge_date  hospital_bill       city blood_pressure  sugar_level  heart_rate
0       P001  Rahul Sharma   45    Male      Diabetes     2026-05-01     2026-05-05          25000      Delhi         140/90          180          88
1       P002   Priya Verma   32  Female  Hypertension     2026-05-02     2026-05-06          18000     Mumbai         150/95          110          92
2       P003    Aman Gupta   60    Male  Heart Disease     2026-05-03     2026-05-10          75000  Bangalore        160/100          140         105
3       P004    Neha Singh   28  Female        Asthma     2026-05-04     2026-05-07          12000    Chennai         120/80           95          80
4       P005     Rohan Das   50    Male      Diabetes     2026-05-05     2026-05-09          30000    Kolkata         145/92          210          90

Data Cleaning Completed

PostgreSQL Connection Successful

Data Loaded into PostgreSQL Successfully
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline\scripts>
```

Step 3 — Apache Kafka Streaming Integration

Apache Kafka was implemented to demonstrate real-time healthcare data streaming concepts.

Activities Performed:

• Kafka Producer implementation

• Kafka Consumer implementation

• Healthcare message streaming

• Real-time ingestion validation

Benefits:

• Event-driven architecture

• Real-time data ingestion

• Scalable streaming platform

• Future-ready healthcare pipeline

The Kafka Producer and Consumer successfully exchanged healthcare records, demonstrating streaming capabilities.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline> python kafka_demo/consumer.py
Waiting for messages...
Received: Patient_ID=101, Disease=Diabetes
```

```
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline> python kafka_demo/producer.py
Message sent successfully!
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline>
```

Figure: Apache Kafka Producer–Consumer Demonstration

Step 4 — Apache Airflow Automation

Apache Airflow was implemented for workflow orchestration and automation.

Features:

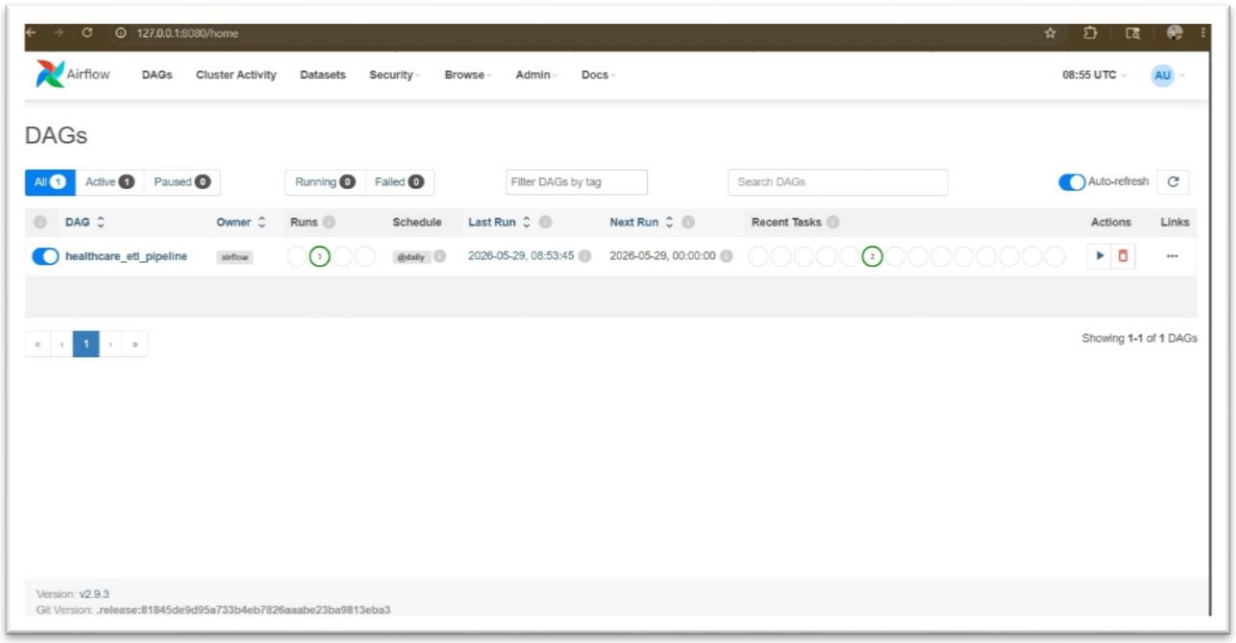
• DAG-based workflow execution

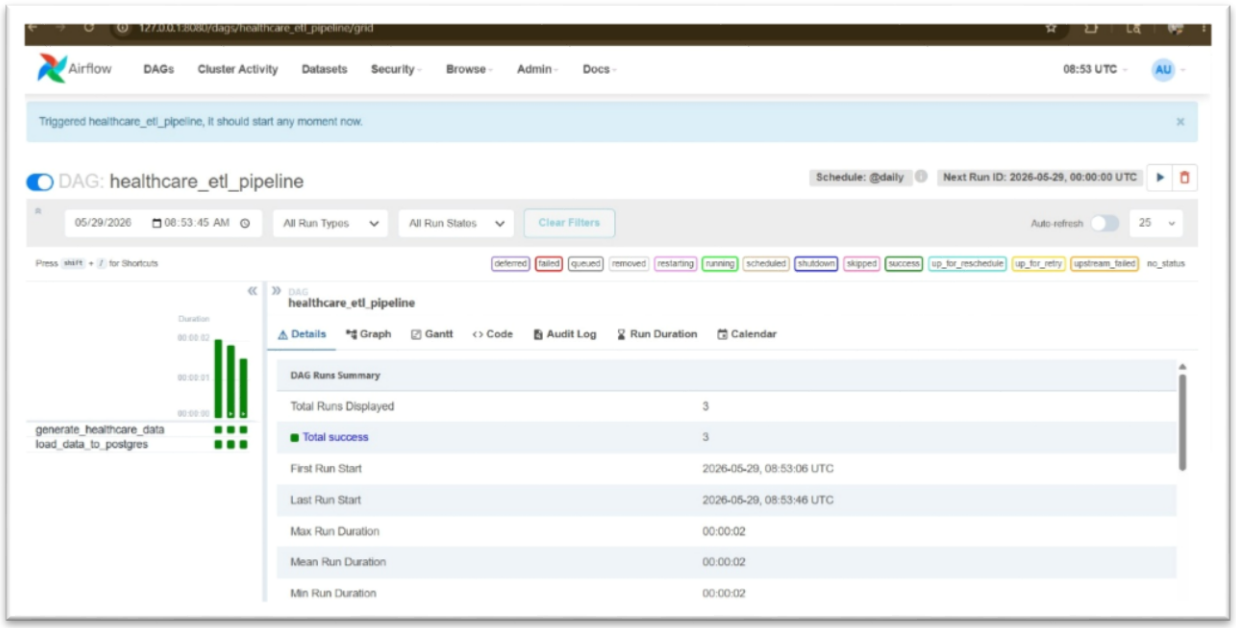
• Automated scheduling

• Monitoring and logging

• Pipeline management

The Airflow DAG successfully automated ETL execution and workflow monitoring.





Step 5 — Databricks & PySpark Big Data Processing

Apache Spark was integrated into the pipeline using PySpark for efficient processing of large healthcare datasets. The PySpark workflows were also validated using Databricks notebooks to demonstrate industry-standard big data processing and analytics capabilities.

Activities Performed:

• Loading healthcare dataset into Spark

• Schema validation

• Record counting

• Disease distribution analysis

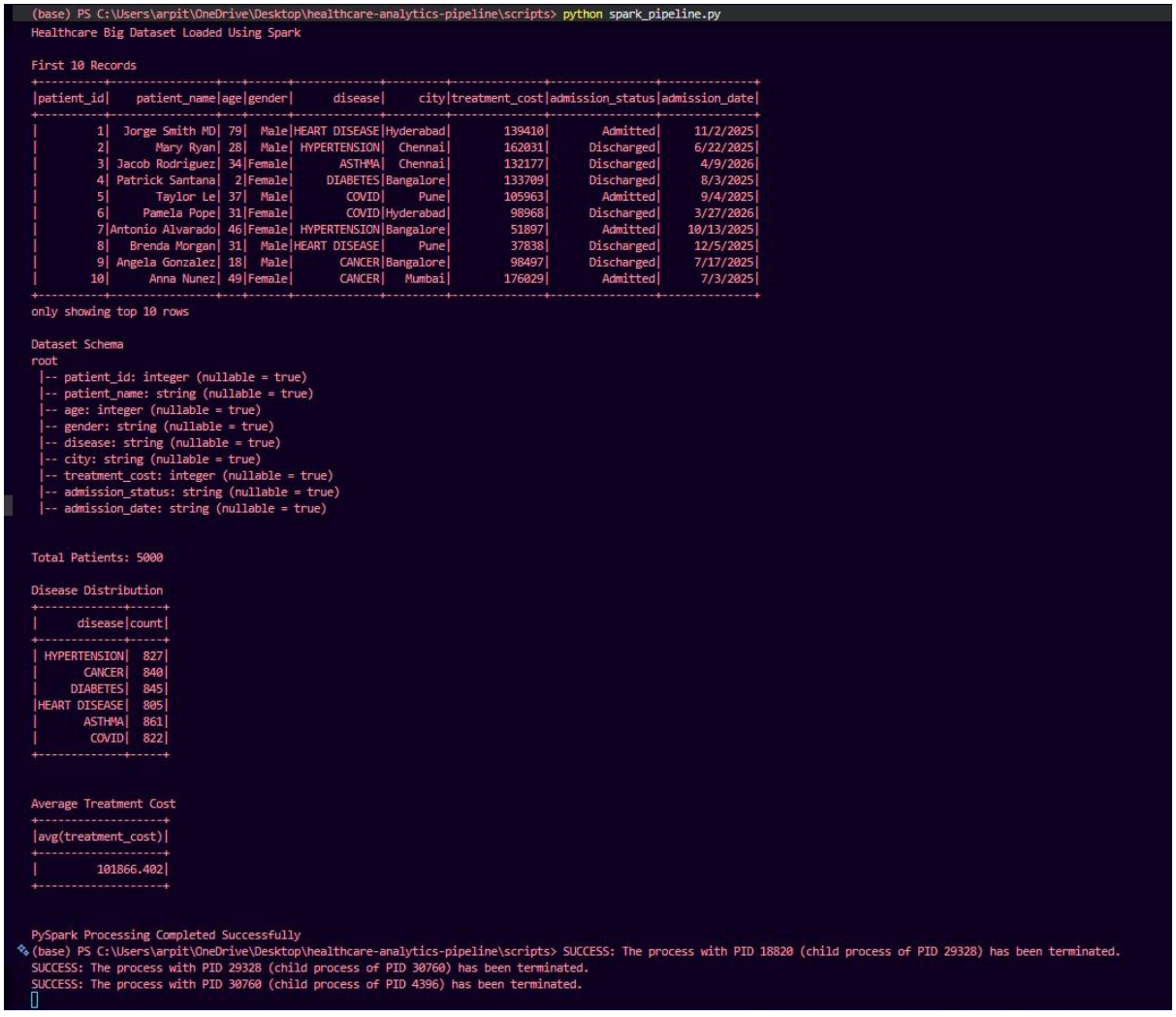
• Treatment cost analysis

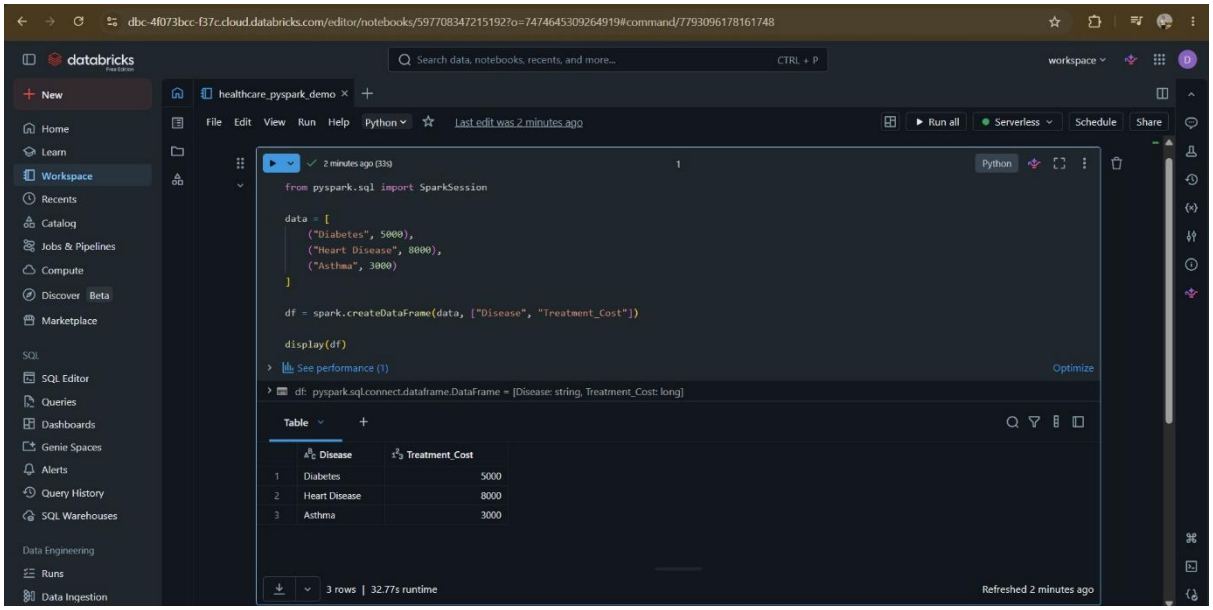
Results Obtained:

• Successfully processed 5000 healthcare records

• Generated disease-wise analytics

• Demonstrated scalable distributed processing





Step 6 — AWS RDS PostgreSQL Integration

AWS RDS PostgreSQL was used as the cloud database for storing processed healthcare records.

Benefits:

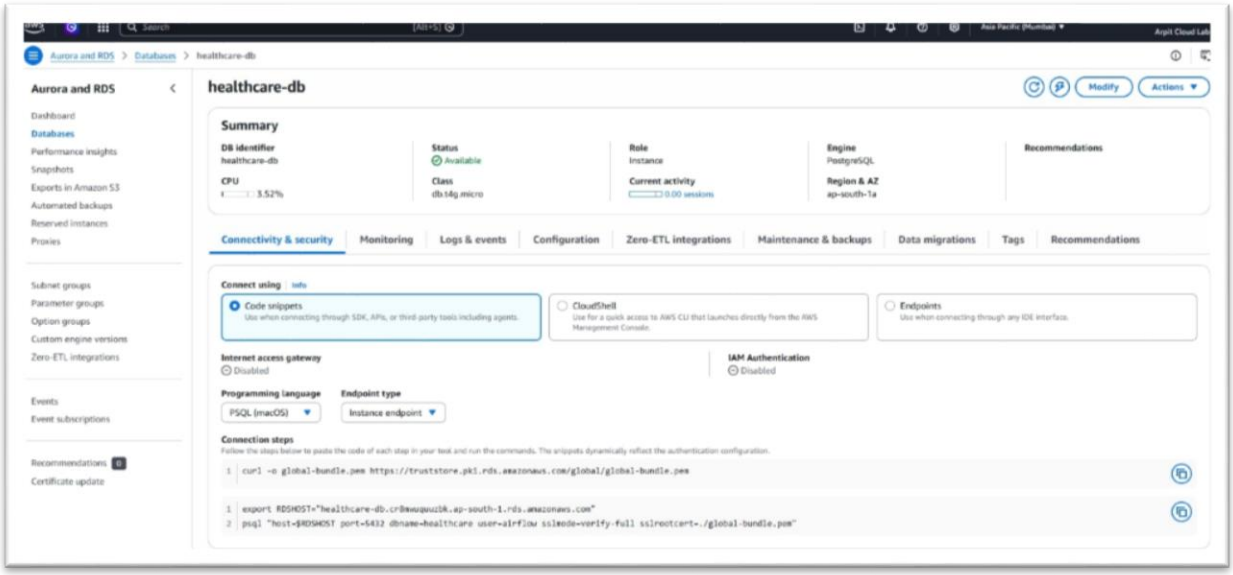
• Managed database service

• Cloud-based accessibility

• Secure data storage

• High availability and reliability

The ETL pipeline successfully loaded healthcare records into AWS RDS PostgreSQL.



Step 7 — Data Warehouse Design

A Star Schema Data Warehouse was implemented to support analytical reporting and dashboard development.

Warehouse Components:

Dimension Tables

• dim_patient

• dim_disease

• dim_date

Fact Table

• fact_patient_visits

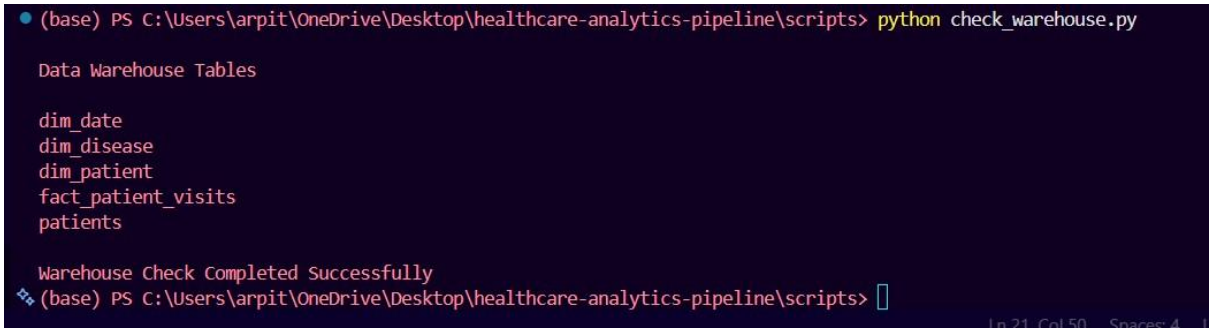
Benefits:

• Faster analytical queries

• Better reporting performance

• Simplified data analysis

• Improved dashboard responsiveness



Step 8 — Interactive Dashboard Development

A Streamlit dashboard was developed for healthcare analytics and reporting.

Dashboard Features:

• Total Patients KPI

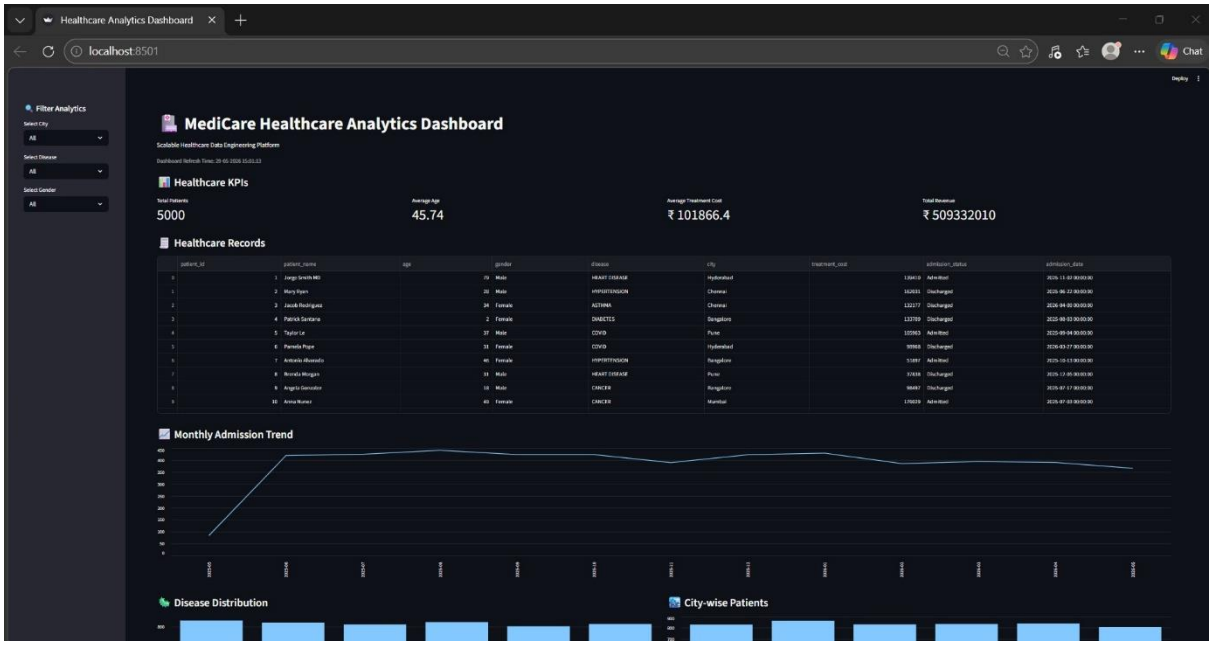
• Average Treatment Cost

• Disease Distribution Analysis

• Patient Data Visualization

• Interactive Reporting

The dashboard provides healthcare insights through a user-friendly interface.



PART 6 — RESULTS

The Healthcare Analytics Pipeline was successfully implemented and validated.

Major Achievements:

• Processed 5000 healthcare records successfully

• Automated ETL workflow using Python

• Integrated AWS S3 cloud storage

• Implemented Apache Airflow automation

• Performed distributed processing using PySpark

• Integrated AWS RDS PostgreSQL

• Designed a Star Schema Data Warehouse

• Developed a Streamlit analytics dashboard

• Implemented machine learning analytics

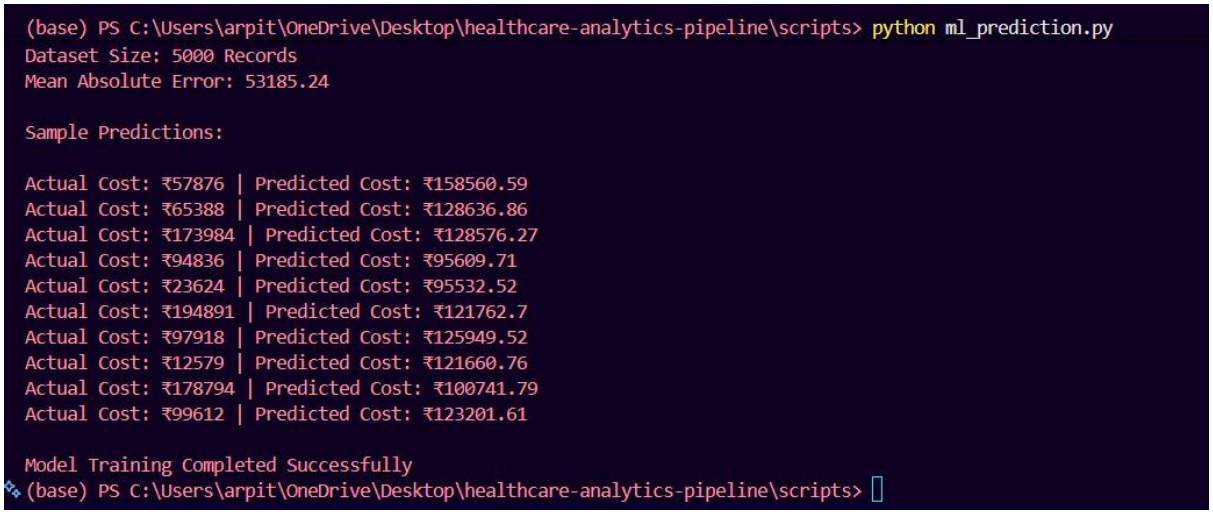
• Implemented Apache Kafka streaming demonstration

• Executed PySpark processing in Databricks

• Implemented GitHub Actions CI/CD workflow

Performance Summary

Component	Status
AWS S3 Integration	Successful
ETL Pipeline	Successful
Airflow Automation	Successful
PySpark Processing	Successful
AWS RDS Integration	Successful
Data Warehouse	Successful
Dashboard Development	Successful
Machine Learning	Successful
Apache Kafka Integration	Successful
Databricks Processing	Successful
GitHub Actions CI/CD	Successful



```
==================================================
EDA REPORT
--------------------------------------------------
(base) PS C:\Users\arpit\OneDrive\Desktop\healthcare-analytics-pipeline\scripts> python eda_analysis.py

Total Records:
5000

Columns:
['patient_id', 'patient_name', 'age', 'gender', 'disease', 'city', 'treatment_cost', 'admission_status', 'admission_date']

Missing Values:
patient_id         0
patient_name       0
age                0
gender             0
disease            0
city               0
treatment_cost     0
admission_status   0
admission_date     0
dtype: int64

Disease Distribution:
disease
ASTHMA          861
DIABETES        845
CANCER          840
HYPERTENSION    827
COVID           822
HEART DISEASE   805
Name: count, dtype: int64

Gender Distribution:
gender
Male      2534
Female    2466
Name: count, dtype: int64

City Distribution:
city
Chennai      863
Mumbai       839
Hyderabad    834
Delhi        830
Bangalore    828
Pune         806
Name: count, dtype: int64

Admission Status Distribution:
admission_status
Admitted      2511
Discharged    2489
Name: count, dtype: int64

Average Treatment Cost:
101866.4

Maximum Treatment Cost:
199989

Minimum Treatment Cost:
5004

EDA Completed Successfully
```

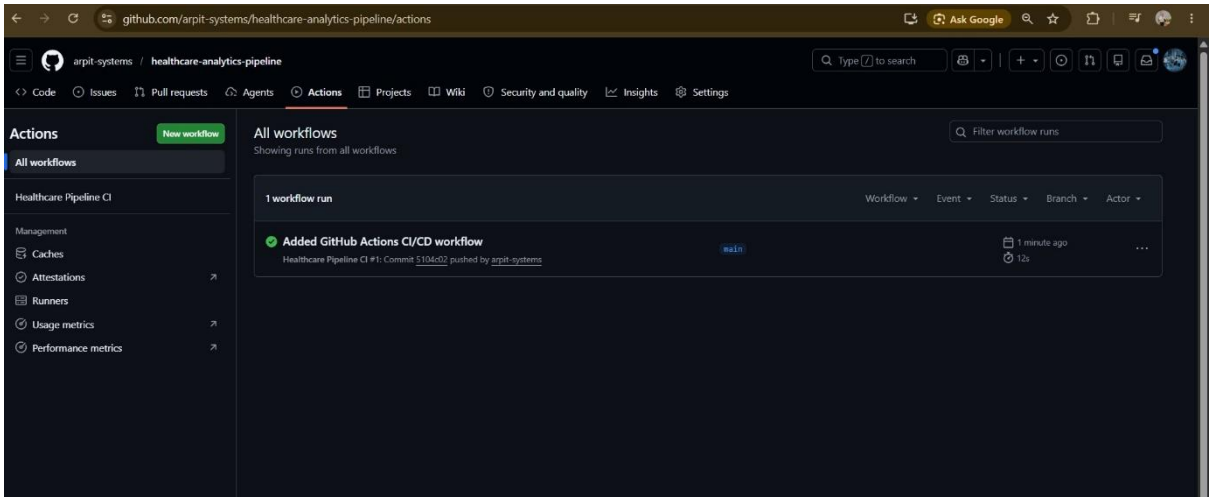


Figure: Databricks Notebook Executing PySpark Processing

PART 7 — COMPARISON AND ANALYSIS

Feature	Traditional Approach	Proposed System
Data Processing	Manual	Automated
Storage	Local Files	AWS Cloud
Workflow Management	Manual	Airflow
Big Data Support	Limited	PySpark
Reporting	Static	Interactive Dashboard
Scalability	Low	High
Analytics	Basic	Advanced
Machine Learning	Not Available	Implemented
CI/CD Automation	Not Available	Implemented

The proposed system significantly improves scalability, automation, reporting capabilities, and analytical performance compared to traditional healthcare data processing approaches.

PART 8 — ARCHITECTURE DIAGRAM

Healthcare Analytics Pipeline Architecture

This architecture represents the complete flow of data from ingestion and processing to analytics, machine learning, and dashboard visualization.

The pipeline is designed using modern Data Engineering principles and integrates cloud storage, workflow automation, big data processing, data warehousing, and predictive analytics into a single platform.

Healthcare data is collected, processed, transformed, and stored through multiple layers before being analyzed and visualized for decision-making purposes.

The architecture ensures scalability, automation, reliability, and efficient reporting while demonstrating the practical application of industry-standard tools and technologies.

The following diagram illustrates the overall architecture and data flow of the Healthcare Analytics Pipeline Project.

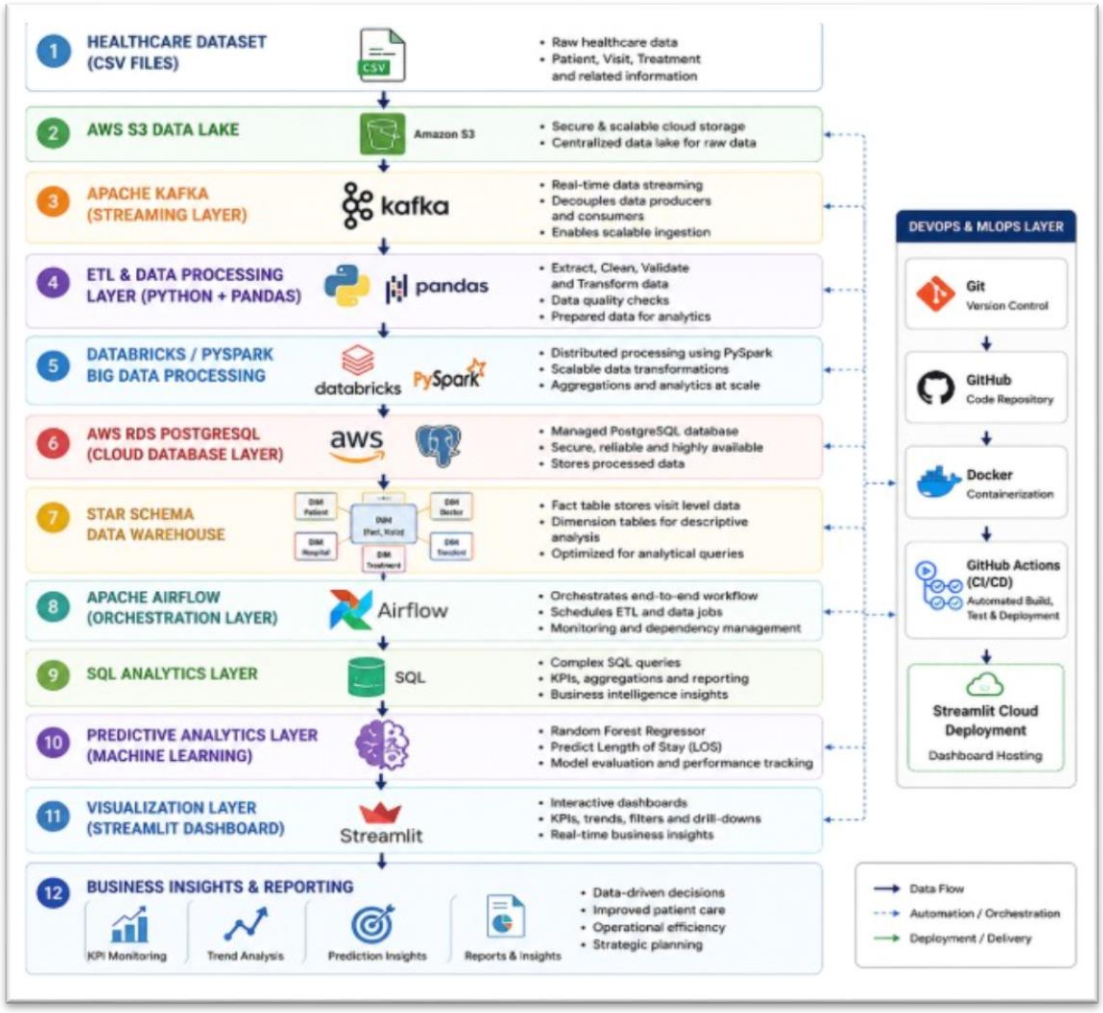


Figure: Healthcare Analytics Pipeline Architecture

PART 9 — DATA GOVERNANCE

Healthcare data requires proper governance, validation, and security controls to ensure data quality and reliability.

Data Governance Measures:

• Validation of incoming healthcare records

• Missing value detection and handling

• Duplicate record identification

• Data consistency checks

• Structured database design

• Secure cloud storage using AWS services

• Controlled access to healthcare information

Benefits:

• Improved data quality

• Reliable analytics

• Better reporting accuracy

• Enhanced data integrity

PART 10 — CONCLUSION

This project successfully demonstrates the implementation of a complete Healthcare Analytics Pipeline for MediCare Hospital.

The system integrates AWS S3, Apache Kafka, Python ETL, Databricks, PySpark, Apache Airflow, PostgreSQL, AWS RDS, Data Warehousing, Machine Learning, Streamlit, Docker, and GitHub Actions CI/CD into a unified analytics platform.

Major Achievements:

• Automated healthcare data processing

• Cloud-based storage and database integration

• Real-time streaming demonstration using Apache Kafka

• Big data processing using Databricks and PySpark

• Interactive dashboard development

• Machine learning implementation

• CI/CD workflow automation using GitHub Actions

• Data warehouse design for analytical reporting

The project demonstrates practical applications of Data Engineering, Cloud Computing, Big Data Processing, Analytics, Machine Learning, and DevOps concepts in a healthcare environment.

PART 11 — FUTURE SCOPE

The Healthcare Analytics Pipeline can be further enhanced using advanced technologies and real-time processing capabilities.

Future Enhancements:

• Advanced machine learning models

• Integration with hospital management systems

• Automated alert generation

• Cloud-native deployment

• Mobile dashboard integration

• Predictive healthcare analytics

• Enterprise-scale implementation

• Snowflake Data Warehouse Integration

• Kubernetes-based Container Orchestration

• Production-grade Apache Kafka streaming architecture.

These improvements can further increase the scalability and analytical capabilities of the platform.

PART 12 — SOURCE CODE STRUCTURE

Project Directory Structure:

healthcare-analytics-pipeline/

├── .github/

| └── workflows/

| └── main.yml

├── dashboards/

| ├── app.py

| └── app_live.py

├── data/

| ├── patients.csv

| └── healthcare_big_data.csv

├── docker_airflow/

| ├── dags/

| | └── healthcare_etl_dag.py

| └── docker-compose.yml

├── docs/

├── kafka_demo/

| ├── producer.py

| └── consumer.py

├── screenshots/

├── scripts/

| ├── etl_pipeline.py

| ├── spark_pipeline.py

| ├── data_warehouse.py

| ├── ml_prediction.py

| └── eda_analysis.py

├── sql/

| └── healthcare_queries.sql

├── weekly_reports/

├── README.md

├── requirements.txt

└── .gitignore

PART 13 — ISSUES FACED & SOLUTIONS

Issue	Solution
Airflow Webserver Not Starting	Reconfigured Docker Compose Setup
AWS RDS Connectivity Issues	Corrected Database Configuration
ETL File Path Errors	Updated Relative File Paths
PySpark Dataset Loading Issues	Modified Dataset References
Dashboard Rendering Issues	Updated Streamlit Components
Data Warehouse Loading Errors	Corrected Table Structure and Schema
Kafka Configuration Issues	Resolved Producer-Consumer Connectivity

The above issues were resolved through testing, debugging, and configuration improvements.

PART 14 — DEMO & LINKS

GitHub Repository

The complete project source code, documentation, ETL scripts, Airflow DAGs, PySpark jobs, SQL queries, data warehouse implementation, and dashboard code are available on GitHub.

GitHub Repository Link:

https://github.com/arpit-systems/healthcare-analytics-pipeline

Live Dashboard

The Streamlit dashboard provides healthcare analytics, KPI monitoring, disease analysis, and interactive reporting.

Dashboard Link:

https://medicare-healthcare-dashboard.streamlit.app

Project Demonstration

Project Workflow:

Healthcare Dataset → AWS S3 → Apache Kafka → Python ETL → Databricks/PySpark → AWS RDS PostgreSQL → Data Warehouse → Apache Airflow → SQL Analytics → Machine Learning → Streamlit Dashboard

The project successfully automates healthcare data processing, analytics, and reporting through an end-to-end data engineering pipeline.

PART 15 — TEAM CONTRIBUTION

This project was completed through the collaborative efforts of all team members. Responsibilities included data collection, data cleaning, ETL development, workflow automation using Apache Airflow, cloud integration, PySpark processing, data warehousing, machine learning, dashboard development, testing, documentation, presentation preparation, and project review.

All team members actively contributed to the successful completion of the Healthcare Analytics Pipeline for MediCare Hospital project.

FINAL NOTE

This report presents the complete implementation of a Healthcare Analytics Pipeline developed using modern Data Engineering technologies. The project demonstrates cloud integration, workflow automation, big data processing, analytics, machine learning, and interactive visualization within a healthcare domain.

The implementation successfully fulfills the objectives of the capstone project and provides a scalable foundation for future healthcare analytics applications.